INFT13-320/73-320

Query Processing and Optimization

Girish Mohata
September Semester, 2002
School of Information Technology
Bond University

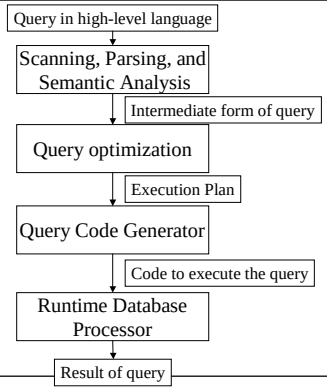
Objectives

- Understand the overall picture of query processing
- Understand what is logical level optimization
- Understand how relational algebra could be rewritten using transformation rules to optimize the query plan at logical level
- Understand the purpose of estimating result sizes and how to estimate them
- Recommended Readings
  - Ramakrishna & Gehrke – Chapter 11, 12, 13 & 14
  - ElMasri & Navathe – Chapter 18

The Big Picture

- We already know:
  - How data is stored in DBMS: Disks, Files, Pages/Blocks, Records, Fields
  - Cost evaluation metrics: Scan, Search, Insert/Delete/Update records
  - Indexing: Conventional Indexes, B+ Tree, Hashing; Costs & Performances
- Next
  - How a query is processed
  - How to estimate costs for query plans and choose the 'cheapest' one
  - Given all these, how to have a good physical design?

Overview of Query Processing



Implementation of Operations

- Selections (σ)
- Joins
- Projections (π)
- Set operations
  - Cartesian Product (×)
  - Union (∪)
  - Intersection (∩)
  - Difference (-)
- Other
  - Duplicate elimination
  - Sorting
  - Aggregations

Sample Database

- Customer (CustomerID, Name, Street, City, State, Zip)
- Rented (CustomerID, FilmID, TapeNum, Date, ReturnDate, AmountPaid, Length, EmpID)
- Film (FilmID, Title, Date, RentalPrice, Distributor, Type)

Selection

- Primary key, point
  $\sigma_{FilmID = 000002}$ (Film)
- Point
  $\sigma_{Title = 'Terminator'}$ (Film)
- Range
  $\sigma_{1 < RentalPrice < 4}$ (Film)
- Conjunction
  $\sigma_{Type = 'M' \wedge Distributor = 'MGM'}$ (Film)
- Disjunction
  $\sigma_{PubDate < 1990 \vee Distributor = 'MGM'}$ (Film)

Selection Strategies

- Linear search
  - Expensive, but always applicable.
- Binary search
  - Applicable only when the file is appropriately ordered.
- Primary hash index search
  - Single record retrieval; does not work for range queries.
  - Retrieval of multiple records.
- Clustering index search
  - Multiple records for each index item.
  - Implemented with single pointer to block with first associated record.

Selection Strategies for Conjunctive Queries

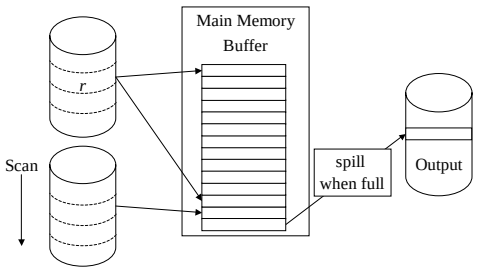
- Use any available indices for attributes involved in simple conditions.
  - If several are available, use the most selective index. Then check each record with respect to the remaining conditions.
- Attempt to use composite indices.
  - This can be very efficient.
- Do intersection of record pointers.
  - If several indices with record pointers are applicable to the selection predicate, retrieve and intersect the pointers. Then retrieve (and check) the qualifying records.
- Disjunctive queries provide little opportunity for smart processing.

Joins

- Sample Joins
  Customer n *Rented*
  Customer n $_{R.EmplId = C.CustomerID}$ *Rented*
- Join Strategies
  - Nested loop join
  - Index/hash-based join
  - Sort-merge join
  - Hash join
- All strategies work on a per block basis (not a per record basis).
- Relation sizes and *join selectivities* always affect the cost of a join.

Nested Loop

- For each block of the outer table, scan the entire inner table.
- Requires quadratic time, O(n^2)



Nested Loop Join

- Exhaustive comparison (ie brute force approach)
- The ordering (outer/inner) of files and allocation of buffer space is important.
  - Desire as few scans as possible of the inner
  - Inner should be largest (in block size)
  - Outer should be smallest (in block size)

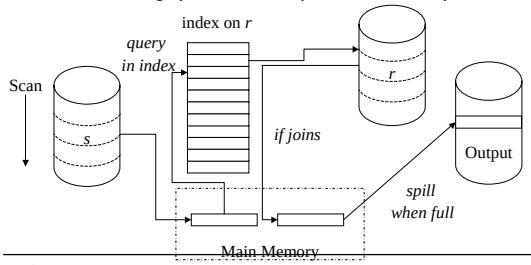
Example of Nested-Loop Join

Customer ⋈ $_{R.EmpId = C.CustomerID}$ *Rented*

- Parameters
  r_{Rented} = 40000 $r_{Customer}$ = 200
  b_{Rented} = 2000 $b_{Customer}$ = 10
  n_B = 6 (size of main memory buffer)
- Algorithm:
  repeat: read (n_B - 2) blocks from outer relation
  repeat: read 1 block from inner relation
  compare tuples
- Cost: $b_{outer} + (\lceil b_{outer} / (n_B - 2) \rceil) \times b_{inner}$
- Rented as outer: 2000 + (2000/4) × 10 = 7000
- Customer as outer: 10 + (10/4) × 2000 = 6010

Index-based Join

- Requires (at least) one index on a join attribute.
- At times, a temporary index is created for the purpose of a join.
- The ordering (outer/inner) of files is important.



Example of Index-based Join

Customer ⋈ $_{R.EmpId = C.CustomerID}$ *Rented*

- Assume that the video store has 10 employees.
  - There are 10 distinct EmpIDs in Rented.
- Assume 1-level sparse index on CustomerID of Customer.
  - Each block contains 100 block pointers.
    - Index is only 1 block (since there are only 10 blocks in customer)
  - Iterate through all 40000 tuples in Rented
    - 2000 disk reads to scan Rented
    - For each Rented tuple, search for matching Customer tuples using index.
      - 0 disk reads for index + 1 disk read for actual data block
    - Cost: 2000 + 40000 × (0+1) = 42000

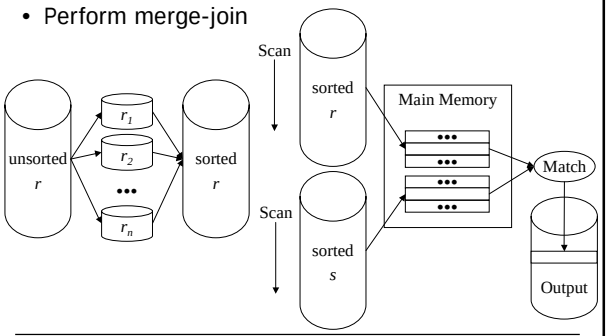
Example of Index-based Join, Cont.

Customer ⋈ $_{R.EmpId = C.CustomerID}$ *Rented*

- There are 10 distinct EmpIDs in Rented.
- Assume 1-level dense index on EmpID of Rented.
  - Each EmpID entry points to a linked list of blocks.
  - Each block in index contains 100 tuple pointers.
    - 40000 / 100 = 400 blocks in index
    - Approximately 400/10 = 40 blocks per EmpID
  - Iterate through all 200 tuples in Customer
    - 10 disk blocks read in scan of Customer
    - For each Customer tuple, search for matching Rented tuples using index
      - Only 10/200 = 5% of customers are also employees
      - Approximately 40000 / 10 = 4000 Rented tuples per EmpID
  - Cost: 10 + (200 × (0.05 × (40 + 4000))) = 40410

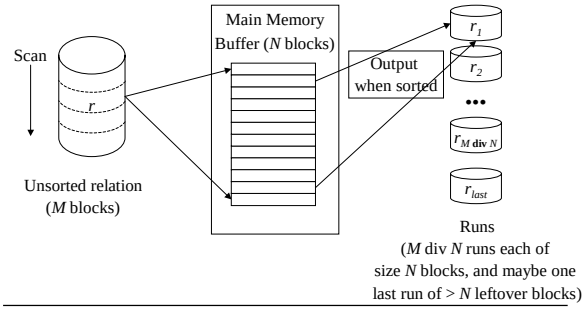
Sort-Merge Join

- Sort each relation using multiway merge-sort
- Perform merge-join



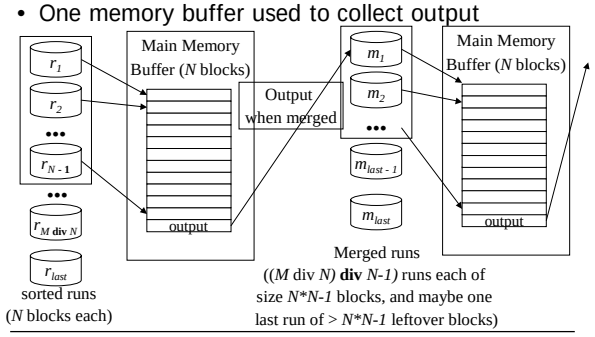
External or Disk-based Sorting

- Relation on disk often too large to fit into memory
- Sort in pieces, called *runs*



External or Disk-based Sorting, Cont.

- Runs are now repeatedly merged
- One memory buffer used to collect output



Example of Sort-Merge Join

- Cost to sort *Rented* relation
  - 1 memory block needed for output buffer in merge steps
  - 2000 reads/2000 writes to generate initial runs
    - 333 runs of 6 blocks each
    - 1 run of 2 blocks
  - 2000 reads/2000 writes for first merge level
    - 66 runs of 30 blocks each
    - 1 run of 20 blocks
  - 2000 reads/2000 writes for second merge level
    - 13 runs of 150 blocks each
    - 1 run of 50 blocks
  - 2000 reads/2000 writes for third merge level
    - 2 runs of 750 blocks
    - 1 run of 500 blocks
  - 2000 reads/2000 writes for last merge level
  - Cost: $5 \times (2000 + 2000) = 20000$

Example of Sort-Merge Join

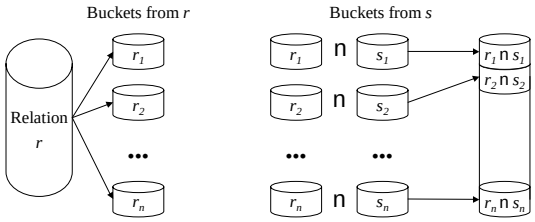
- Cost to sort *Customer* relation
  - 10 reads/10 writes to generate initial runs
    - 1 run of 6 blocks
    - 1 run of 4 blocks
  - 10 reads/10 writes for single merge level

  Cost: $2 \times (10 + 10) = 40$
- Cost for merge-join
  - Cost to scan sorted Customer + cost to scan sorted Rented

  Cost: 10 + 2000 = 2010
- Total cost: 20000 + 40 + 2010 = 22050

Hash Join

- Hash each relation on the join attributes
- Join corresponding buckets from each relation



Example of Hash Join

- Cost to hash Rented
  - One memory block needed for input buffer in hash steps
  - 2000 reads/2000 writes to perform first hash step
    - 5 buckets of 400 blocks each
  - 2000 reads/2000 writes to perform second hash step
    - 25 buckets of 80 blocks each
  - 2000 reads/2000 writes to perform third hash step
    - 125 buckets of 16 blocks each
  - 2000 reads/2000 writes to perform fourth hash step
    - 500 buckets of 4 blocks each

  Cost: $5 \times (2000 + 2000) = 20000$

Example of Hash Join

- Cost to hash Customer
  - 10 reads/10 writes for first hash step
    - 5 buckets of 2 blocks
  - 10 reads/10 writes for second hash step
    - 10 buckets of 1 block

  Cost: $4 \times 10 = 40$
- Cost to join buckets
  - Cost to scan hashed Rented + cost to scan hashed Customer

  Cost: 2000 + 10 = 2010

Total cost: 20000 + 4+ 2010 = 22050

Cost and Applicability of Join Strategies

- Sort-merge join
  - Requires that the files are sorted on the join attributes.
  - Sorting can be done for the purpose of the join.
  - A variation is also applicable when secondary indices are available instead.
- Hash join
  - Requires good hashing functions to be available.
  - Performance best if smallest relation fits in memory.

Other Operations

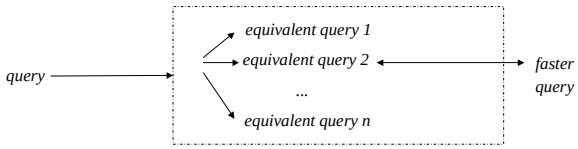
- Projection
  - Example: π_{Name} (*Customer*)
  - No duplicates
  - Duplicates
    - Sort and scan
    - Hashing and checking of buckets
- Cartesian products
  - Inherently expensive
- Union, intersection, and difference
  - Sort and scan
  - Hashing and checking of buckets

Combined Operators

- Often algorithms for combined operations exist. This code is commonly generated dynamically.
  - For example, during joins, selections and projections may be performed on the fly
  - List the titles of expensive films that have been reserved.
  - $\pi_{Title}(\sigma_{AmountPaid > 4}(Film \cap Rented))$
- Adds complexity to the processor.
- Saves the writing and subsequent reading of temporary results.

Query Optimization

- Transform query into faster, equivalent query



- Heuristic (logical) optimization
  - Query tree (relational algebra) optimization
  - Query graph optimization
- Cost-based (physical) optimization

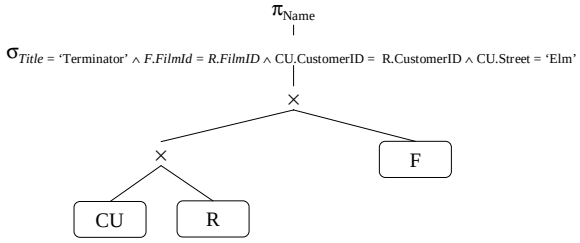
Query Tree Optimization Example

- What are the names of customers living on Elm Street who have checked out “Terminator”?
- SQL query:

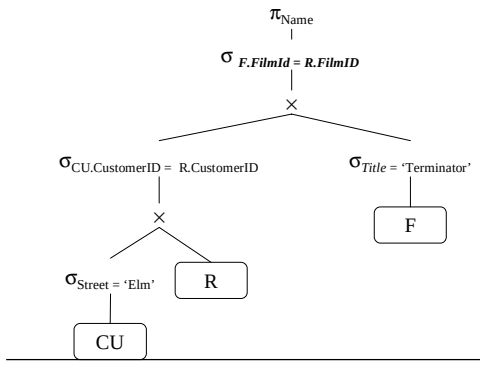
```
SELECT Name
FROM  Customer CU, Rented R, Film F
WHERE Title = 'Terminator'
AND F.FilmId = R.FilmID
AND CU.CustomerID = R.CustomerID
AND CU.Street = 'Elm'
```

- Query is ‘parsed’ to create query tree

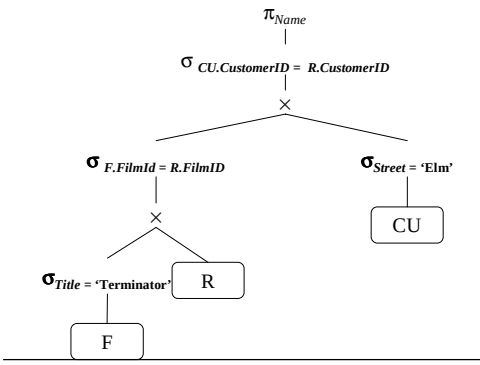
Canonical Query Tree



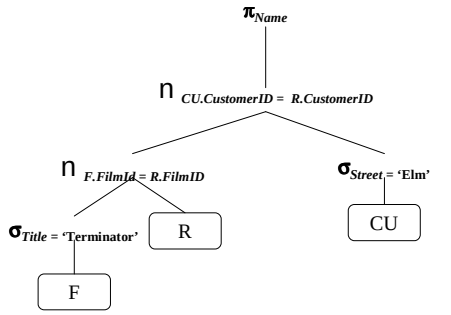
Apply Selections Early



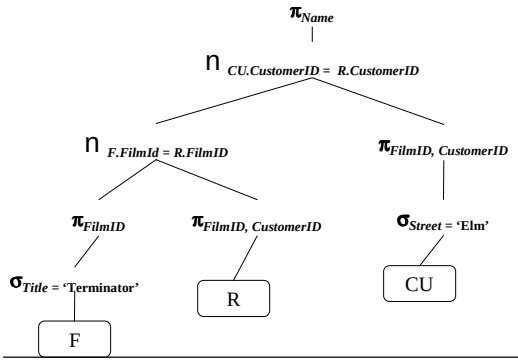
Apply More Restrictive Selections Early



Form Joins



Apply Projections Early



Transformation Rules

1 Cascade of σ

$$\sigma_{c_1 \wedge c_2 \wedge \ldots \wedge c_n}(R) = \sigma_{c_1}(\sigma_{c_2}(\ldots(\sigma_{c_n}(R))\ldots))$$

2 Commutativity of σ

$$\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R))$$

3 Commuting σ with π (only if *c* involves solely attributes in *L*)

$$\pi_L(\sigma_c(R)) = \sigma_c(\pi_L(R))$$

4 Commuting σ with n (only if *c* involves solely attributes in *R)*

$$\sigma_c(R \text{ n } S) = \sigma_c(R) \text{ n } S$$

Transformation Rules (cont)

5 Commuting σ with set operations (where θ is one of ∪, ∩, or -)

$$\sigma_c(R \ \theta \ S) = \sigma_c(R) \ \theta \ \sigma_c(S)$$

6 Associativity of r , ∪, ∩, and -

$$(R \ \theta \ S) \ \theta \ T = R \ \theta \ (S \ \theta \ T)$$

7 Commutativity of ∪, ∩, and × (where q is one of ∪, ∩ and ×)

$$R \ \theta \ S = S \ \theta \ R$$

8 Cascade of π

$$\pi_{L_1}(\pi_{L_2}\ldots(\pi_{L_n}(R))\ldots)) = \pi_L(R)$$

Transformation Rules (cont)

9 Commuting π with n (or ×)

$$\pi_L(R \bowtie_c S) = \pi_L((\pi_{A_1 \ldots A_n, A_{n+1} \ldots A_{n+k}}(R)) \bowtie_c (\pi_{B_1 \ldots B_n, B_{n+1} \ldots B_{n+k}}(S)))$$

- Where c involves all attributes A and B and L includes all and $A_1 \ldots A_n$ and $B_1 \ldots B_m$

0 Commuting π with ∪

$$\pi_L(R \cup S) = (\pi_L(R)) \cup (\pi_L(S))$$

Transformation Algorithm Outline

Step 1: Decompose σ operations.
- Use Rule 1.

Step 2: Move σ as far down the query tree as possible.
- Use Rules 2, 3, 4, and 5.

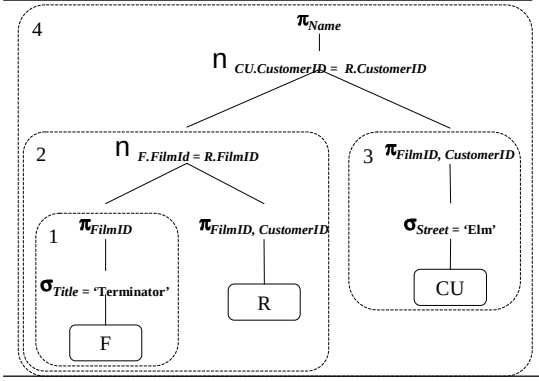
Step 3: Rearrange leaf nodes to apply the most restrictive σ operations first.
- Use Rules 6, 7, and 8.

Step 4: Form joins from × and subsequent σ operations.

Step 5: Decompose π and move down the query tree as far as possible.
- Use Rules 3, 9, 10, and 11.

Step 6: Identify candidates for combined operations.

Example: Identify Combined Operations



Heuristic Query Optimization Summary

- Generate initial query tree from SQL statement.
- Transform query tree into more efficient query tree, via a series of tree modifications, each of which hopefully reduces the execution time.
- Single query tree results.

Cost-Based Optimization

- Use transformations to generate multiple candidate query trees from the canonical query tree.
- Statistics on the inputs to each operator are needed.
  - Statistics on leaf relations are stored in the system catalog.
  - Statistics on intermediate relations must be estimated; most important is the relations' cardinalities.
- Cost formulas estimate the cost of executing each operation in each candidate query tree.
  - Parameterized by statistics of the input relations.
  - Also dependent on the specific algorithm used by the operator.
  - Cost can be CPU time, I/O time, communication time, main memory usage, or a combination.
- The candidate query tree with the least total cost is selected for execution.

Relevant Statistics

- Per relation
  - Tuple size
  - Number of tuples (records): *r*
  - Load factor (fill factor), percentage of space used in each block
  - Blocking factor (number of records per block): *bfr*
  - Relation size in blocks: *b*
  - Relation organization
  - Number of overflow blocks

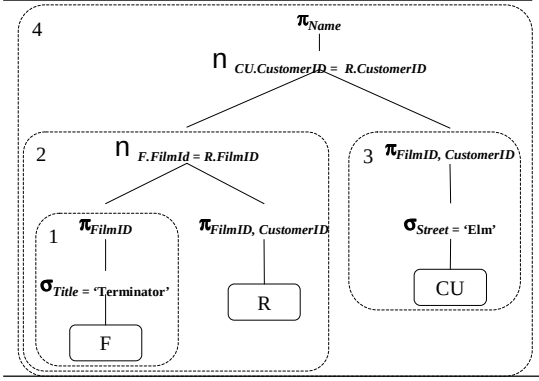
Relevant Statistics (cont)

- Per attribute
  - Attribute size and type
  - Number of distinct values for attribute A: d_A
  - Selection cardinality: s_A
  - Probability distribution over the values
  - Representation, e.g., compressed
- Selection cardinality specifies the average size of $\sigma_{A = a}(R)$ for an arbitrary value *a*.
  - Could be maintained for the "average" attribute value, or on a per-value basis, as a histogram.

Relevant Statistics (cont)

- Per Index
  - Base relation
  - Indexed attribute(s)
  - Organization, e.g., B-Tree, Hash, ISAM
  - Clustering index?
  - On key attribute(s)?
  - Sparse or dense?
  - Number of levels (if appropriate)
  - Number of first-level index blocks: b_1
- General
  - Available main memory blocks: *N*

Cost Estimation Example



Operation 1: σ followed by a π

- Statistics
  - Relation statistics: $r_{Film} = 5{,}000$ $b_{Film} = 50$
  - Attribute statistics: $s_{Title} = 1$
  - Index statistics: Secondary Hash Index on *Title*.
- Result relation size: 1 tuple.
- Operation: Use index with 'Terminator', then project on *FilmID*. Leave result in main memory (1 block).
- Cost (in disk accesses): $C_1 = 1 + 1 = 2$

Operation 2: n followed by a π

- Statistics
  - Relation statistics: $r_{Rented} = 40{,}000$ $b_{Rented} = 2{,}000$
  - Attribute statistics: $s_{FilmID} = 8$
  - Index statistics: Secondary B-Tree Index for *Rented* on *FilmID* with 2 levels.
- Result relation size: 8 tuples.
- Operation: Index join using B-Tree, then project on *CustomerID*. Leave result in main memory (one block).
- Cost: $C_2 = 1 + 1 + 8 = 10$

Operation 3: σ followed by a π

- Statistics
  - Relation statistics: $r_{Customer} = 200$ $b_{Customer} = 10$
  - Attribute statistics: $s_{Street} = 10$
- Result relation size: 10 tuples.
- Operation: Linear search of *Customer*. Leave result in main memory (one block).
- Cost: $C_3 = 10$

Operation 4: n followed by a π

- Operation: Main memory join on relations in main memory.
- Cost: C_4 = 0

Total cost: $C = \sum_{i=1}^{4} C_i = 2+10+10+0 = 20 \approx 200 msec$

Comparison

- Heuristic query optimization
  - Sequence of single query plans
  - Each plan is (presumably) more efficient than the previous.
  - Search is linear.
- Cost-based query optimization
  - Many query plans generated.
  - The cost of each is estimated, with the most efficient chosen.
  - Search is multi-dimensional.

Physical Design: Understanding Workload

- What are the most important queries and how often they arise? For each query, what relation does it access, which attributes are retrieved and which attributes are involved in selection/join conditions?
- What are the most important updates and how often they arise? For each update, what is the type of update, which attribute are involved and which attributes are affected?

Physical Design: Decisions to Make

- What indexes should we create?
  - Which relations should have indexes?
  - Which attribute (s) in the relation should be the search key?
  - Should we have several indexes?
- For each index, what kind of index it should be?
  - B+ Tree ? Hash? Conventional?

How to Choose Indexes?

- One idea: consider the most important queries in turn; consider the best plan using current available indexes first and see if a better plan is possible with an additional index. If so, create the additional index.
- Attributes in WHERE clauses are candidates for index search keys
  - Equality query suggests hash index
  - Range query suggests B+ Tree index
- Choose indexes that would benefit as many queries as possible
- *Multi- attribute search keys* index could be considered when a WHERE clause contains several conditions.
- Always bear in mind index performances and costs: can make queries faster and updates slower; additional disk overheads

Choice of Indexes: Examples

Example 1

SELECT E.ename, D.mgr
FROM Employee e, Dept D
WHERE D.dept_name= 'TOY'
AND E.dept_no=D.dept_no

- Hash index on D.dept_name allows for faster search
- Hash index on E.dept_no allows for faster join
- What if WHERE clause contains additional " E.age=25"? We can use index on E.age and retrieve Employee tuples using this index and then join with Dept. The index on E.dept_no may be not necessary

Example 2

SELECT E.ename, D.mgr
FROM Employee e, Dept D
WHERE E.sal BETWEEN 10000
AND 20000
ANDE.hobby='Soccer'
AND E.dept_no=D.dept_no

- Hash index on D. dept_no allows for faster join
- B+ Tree index on E.sal or hash index on E.hobby. Only one of these two is needed and choice depends on the selectivity of conditions; normally equality selections are more selective than range selections.

Choice of Indexes: Examples (Cont.)

Consider a GROUP BY query example:

```
SELECT E.dept_no, COUNT (*)
FROM Employee E
WHERE E.age>25
GROUP  BY E.dept_no
```

If many tuples have E.age>25, using E.age index and sorting the retrieved tuples may be costly.

- Using clustered E.dept_no index may be better

Choice of Indexes: Examples (Cont.)

Multi-attribute search keys index

- To retrieve Employee records with *age=30* AND *sal=40000*, an index on *<age, sal>* would be better than an index on *age* or an index on *sal*.
- If condition is *30<age<40* AND *30000<sal<40000*, then we can have a B+ Tree index on *<age, sal>* or *<sal, age>*
- If condition is *age=40* AND *30000<sal<40000*, then a B+ Tree index on *<age, sal>* is better than an index on *<sal, age>*

Query Tuning

- Query optimization is a very complex task.
  - Combinatorial explosion.
  - The task is to find one good query evaluation plan, not the best one.
- No optimizer optimizes all queries adequately.
- There is a need for query tuning.
  - All optimizers differ in their ability to optimize queries, making it difficult to prescribe principles.
- Having to tune queries is a fact of life.
  - Query tuning has a localized effect and is thus relatively attractive.
  - It is a time-consuming and specialized task.
  - It makes the queries harder to understand.
  - However, it is often a necessity.
  - This is not likely to change any time soon.

Minimizing DISTINCTs

- We have seen that sorting is expensive; it should thus be avoided whenever possible.
- Sample database

  `Emp (ssn, name, manager, dept, salary)`
  - clustering index on ssn
  - non-clustering indices on name, dept
  - ssn and name are keys

  `Stud (ssn, name, course, grade)`
  - clustering index on ssn
  - non-clustering indices on name
  - ssn and name are keys

  `Techdept (dept, manager, location)`
  - clustering index on dept
  - dept is a key

Minimizing DISTINCTs (cont)

- Consider a query

```
SELECT DISTINCT ssn
FROM    emp
WHERE   dept= 'Information Systems'
```

- Some systems may apply duplicate elimination, although this is not necessary.
- DISTINCTS can be avoided more generally. Define:
  - A table T is *privileged* if the query result contains a key of T.
  - A table R1 *reaches* another table R2 if R1 is equality-joined with R2 on a key of R1. (Reaches is transitive.)
- With these two definitions, DISTINCT can be avoided when
  - Every table mentioned in the SELECT clause is privileged.
  - Every unpriviledged table reaches at least one privileged table.

Minimizing DISTINCTs (cont)

- Examples:

```
SELECT ssn
FROM   emp, teachdept
WHERE  emp.dept = teachdept.dept
```

  - Table *emp* is priviledged, and table *techdept* reaches *emp*, so no duplicates can occur.

```
SELECT ssn
FROM   emp, teachdept
WHERE  emp.manager = teachdept.manager
```

  - Now techdept no longer reaches emp, so duplicates may occur.

```
SELECT ssn, teachdept.dept
FROM   emp, teachdept
WHERE  emp.manager = teachdept.manager
```

  - Now, both tables are priviledged, so no duplicates can occur.

Unnesting Nested Queries

- Uncorrelated sub-queries with aggregates.
  - Most systems would compute the average only once.

```
SELECT ssn
FROM   emp
WHERE  salary > (SELECT AVG(salary) FROM emp)
```

- Uncorrelated sub-queries without aggregates.

```
SELECT ssn
FROM   emp
WHERE  dept IN (SELECT dept FROM techdept)
```

  - Some systems may not use emp's index on dept, so a transformation is desirable.

```
SELECT ssn
FROM   emp, techdept
WHERE  emp.dept = techdept.dept
```


Unnesting Nested Queries (cont)

- Watch out for duplicates! Consider a query and its rewritten counterpart.

```
SELECT AVG(salary)
FROM  emp
WHERE manager IN (SELECT manager FROM techdept)
```

  - Unnested version, with problems:

```
SELECT AVG(salary)
FROM  emp, techdept
WHERE emp.manager = techdept.manager
```

  - This query may yield wrong results! A solution:

```
SELECT DISTINCT (manager) INTO temp
FROM techdept

SELECT AVG(salary)
FROM  emp, temp
WHERE emp.manager = temp.manager
```


Tuning Issues

- Optimizers (eg Ingres and Oracle) are typically unable to optimize queries with multiple joins (more than, eg 5-10 joins).
- Use temporary tables can yield huge speed-ups.
- Express joins on clustering indices, if possible.
  - Example: Join of emp and student on ssn rather than on name.
- Many similar queries may repeat work.
- Compute a temporary table with a common sub-result.
- Views do not improve performance, but may lead to poor performance.

Summary

- Query optimization is the heart of a relational DBMS.
- Heuristic optimization is more efficient to generate, but may not yield the optimal query evaluation plan.
- Cost-based optimization relies on statistics gathered on the relations.
- Until query optimization is perfected, query tuning will be a fact of life.

Acknowledgements

- Curtis Dyreson
- Peter Stewart
- Ramakrishna & Gehrke